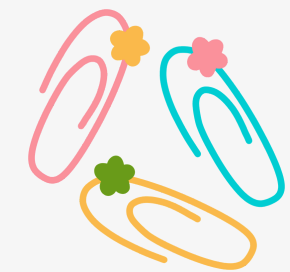


CP353201 Software Quality Assurance

ภาคเรียนที่ 1 ปีการศึกษา 2569

การศึกษา ALGORITHM สำหรับการทดสอบซอฟต์แวร์ และการออกแบบ PROMPT สำหรับ AI

การเปรียบเทียบประสิทธิภาพระหว่าง AUTOMATED TEST GENERATION และ AI-ASSISTED TEST GENERATION บน DEFECTS4J



เสนอ
พศ. ดร.ชิตสุรา สุ่มเล็ก

สมาชิกกลุ่ม

673380051-7 พรชพร มุสันเทียะ
673380065-6 สรวิชญ์ ทักตร

673380052-5 พยดา เกษมาลา
673380280-2 ปิยพล ตุ่นป่า

ALGORITHMS & AI-ASSISTING TOOLS

4 แนวทางที่ใช้ในการทดลอง

GENETIC ALGORITHM (GA)

EvoSuite 1.2.0 Automated Test Case Generation

1

Feedback-directed Random Testing (FRT)

Randoop 4.3.4 Automated Test Sequence Generation

2

DEEPSEEK-V4-FLASH

DeepSeek V4 Flash AI-Assisted Unit Test Generation

3

GEMINI 3.8 FLASH

Gemini 3.8 Flash AI-Assisted Unit Test Generation

4



DEFECTS4J DATASET

ชุดข้อมูลสำหรับการทดสอบซอฟต์แวร์ภาษา JAVA ที่รวบรวมโปรเจกต์ซึ่งมี BUG ที่ทราบอยู่แล้ว

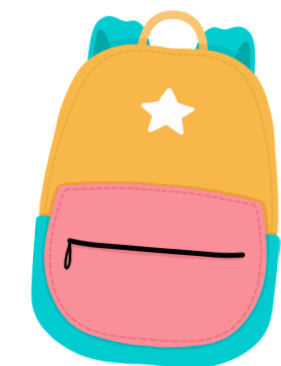
17
Project

854
Bug IDs

1068
Class

ใช้ BUGGY VERSION สำหรับเป็น TARGET ในการ GENERATE TEST CASE

ใช้ FIXED VERSION และ BUGGY VERSION ในการตรวจสอบผลลัพธ์ของ TEST CASE จากเครื่องมือทั้ง 4



GENETIC ALGORITHM (GA)

หลักการทำงาน

- สุ่มลำดับคำสั่งและประเมินค่าความเหมาะสม จากเป้าหมาย Coverage
- คัดเลือกชุดทดสอบที่มีค่า Test Case ผ่านสูงที่สุด
- สุ่มดัดแปลงโค้ดทดสอบเพื่อเจาะเงื่อนไขใหม่ๆ
- เงื่อนไขหยุด: วนรอบวิวัฒนาการไปเรื่อยๆ จนกว่าจะได้ Coverage ครบ 100% ตามเป้าหมาย หรือหมดเวลาค้นหา

ข้อดี

ใช้ Fitness นำทางการค้นหา GA ไม่ได้สุ่ม Input แบบไม่มีทิศทาง แต่จะประเมิน Test Case และนำ Test Case ที่มี Fitness ดีมาต่อยอดด้วย Selection, Crossover และ Mutation ทำให้สามารถค้นหา Input ที่มีโอกาสเข้าถึงส่วนของโปรแกรมที่ยากต่อการทดสอบ และช่วยเพิ่ม Code Coverage ได้อย่างเป็นระบบ

FEEDBACK-DIRECTED RANDOM TESTING

หลักการทำงาน

- สุ่ม เรียก constructor, method และค่า argument มาต่อกันเป็นลำดับคำสั่ง
- รับทันที เพื่อดูผลลัพธ์ (feedback)
- ผ่านปกติ → เก็บเข้า pool เพื่อสุ่มต่อทอดให้ยาวขึ้น
- พัง/error → หยุดขยาย แล้วบันทึกเป็นเคสที่พบบัก
- อินพุตไม่ถูกต้อง → คัดทิ้ง

ข้อดี

ใช้ผลลัพธ์ย้อนกลับ (Feedback) นำทางทันที: ต่างจากการสุ่มแบบทั่วไป (Pure Random) ตรงที่เมื่อสุ่มสร้างลำดับคำสั่งเสร็จจะ "รันทดสอบทันที" เพื่อดูพฤติกรรมของโค้ดก่อนตัดสินใจขึ้นถัดไป ทำให้เหมาะกับ regression test

DEEPSEEK V4 FLASH & GEMINI 3.8 FLASH

4 แนวทางที่ใช้ในการทดลอง

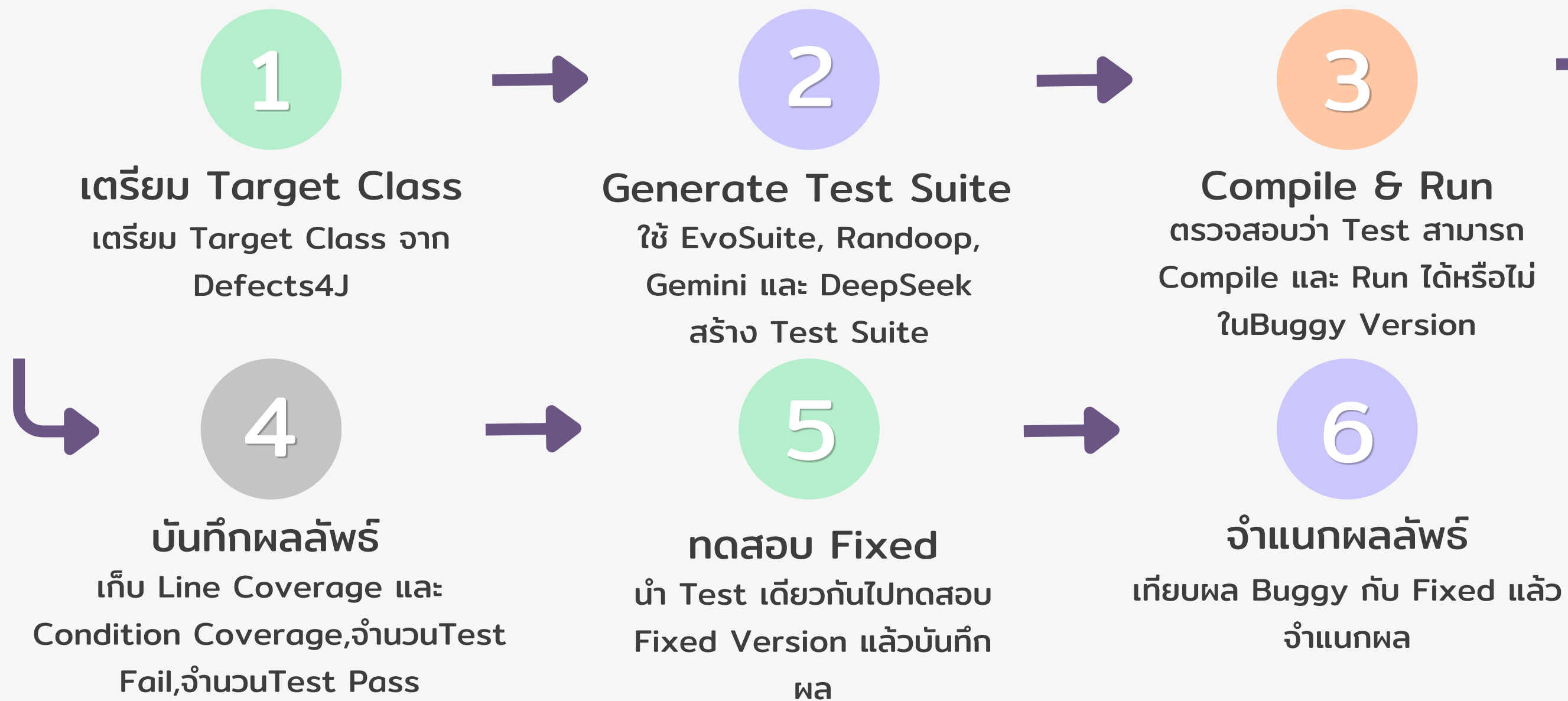


การออกแบบPrompt

- ห้ามใช้ไลบรารีภายนอก โดยสั่งให้ใช้เฉพาะ Plain Java Objects ร่วมกับ assertion มาตรฐานเพื่อป้องกัน compile error จาก dependency ที่ไม่มีอยู่
- สั่งให้ครอบคลุม Boundary value, ค่าว่าง/Null รวมถึงกรณี Error/Exception path เหมือนกันทั้งสองโมเดล
- กำหนดให้โมเดลสร้างโค้ดที่สมบูรณ์และพร้อมคอมไพล์ (Runnable & Complete Code)
- กำหนดให้ใช้ JUnit 4 และ systax Java 7 เพื่อให้สอดคล้องกับโปรเจกต์ของ Defects4J
- กำหนดให้ package ของไฟล์เทสต้องตรงกับ source code ต้นทางเพื่อการเข้าถึง scope และคอมไพล์ได้อย่างถูกต้อง

วิธีการทดลอง

ตั้งแต่เตรียมข้อมูลจนถึงการเปรียบเทียบผล



EXPERIMENTAL CONFIGURATION

การกำหนดค่าที่ใช้เพื่อให้การทดลองทำซ้ำและเปรียบเทียบได้

Genetic Algorithm (GA)

- EvoSuite 1.2.0 : STANDARD_GA
- Coverage: LINE + CONDITION
- Search Budget: 60 sec
- Random Seed: 0,20260918
- 2 Round

Gemini

- Model: Gemini 3.8 Flash
- KKU GenAI API
- Input: Prompt + Source Code
- 1 Round

Feedback-directed Random Testing (FRT)

- Randoop 4.3.4
- Time Budget: 60 sec
- Timeout: 600 sec
- Random Seed: 0,20260928
- 2 Round

DeepSeek

- Model: DeepSeek V4 Flash
- KKU GenAI API
- Input: Prompt + Source Code
- 1 Round

EVALUATION METRICS

ตัวชี้วัดที่ใช้ประเมิน TEST SUITE ที่สร้างได้



1. MEASURED COVERAGE

ค่าเฉลี่ยที่นับ LINE /CONDITION COVERAGE แต่ละ BUG ID ที่สามารถวัดค่าได้จริง

- MEAN LINE COVERAGE
- MEAN CONDITION COVERAGE

2. OVERALL COVERAGE

ค่าเฉลี่ยที่นับ LINE /CONDITION COVERAGE BUG ID ทั้งหมดโดยกำหนด COVERAGE ที่ไม่สามารถรันทดสอบเพื่อวัดค่าได้เป็น 0%

- MEAN LINE COVERAGE (OVERALL)
- MEAN CONDITION COVERAGE (OVERALL)

หมายเหตุ:

- วัดผล Coverage จากทุก Target Class ของแต่ละ BUG ID ในเวอร์ชัน Buggy
- Line /CONDITION COVERAGE ต่อ 1 BUG ID = Line /CONDITION ที่ถูกรัน ÷ Line /CONDITION ทั้งหมด ของทุก TARGET CLASS รวมกัน
- ไม่นับ BUG ID ที่รันและวัด CONDITION COVERAGE สำเร็จ แต่ได้ผล CONDITION เป็น 0

TEST RESULT CLASSIFICATION

จำแนกผลการทดสอบตามความสามารถในการตรวจพบบั๊ก 1 BUG ID ต่อ 1 สถานะ



REVEALING

BUGGY FAIL + FIXED PASS
→ ตรวจพบ BUG

NOT REVEALING

รันได้ แต่ไม่สามารถเปิดเผย BUG

INCONCLUSIVE

FIXED FAIL แม้ BUGGY จะ
PASS หรือ FAIL
ผลการทดสอบไม่สามารถสรุปได้

NOT AVAILABLE

บางชุดทดสอบหรือทั้งหมด ของ BUG ID นั้นๆ
COMPILE / RUN / COVERAGE ไม่สำเร็จ

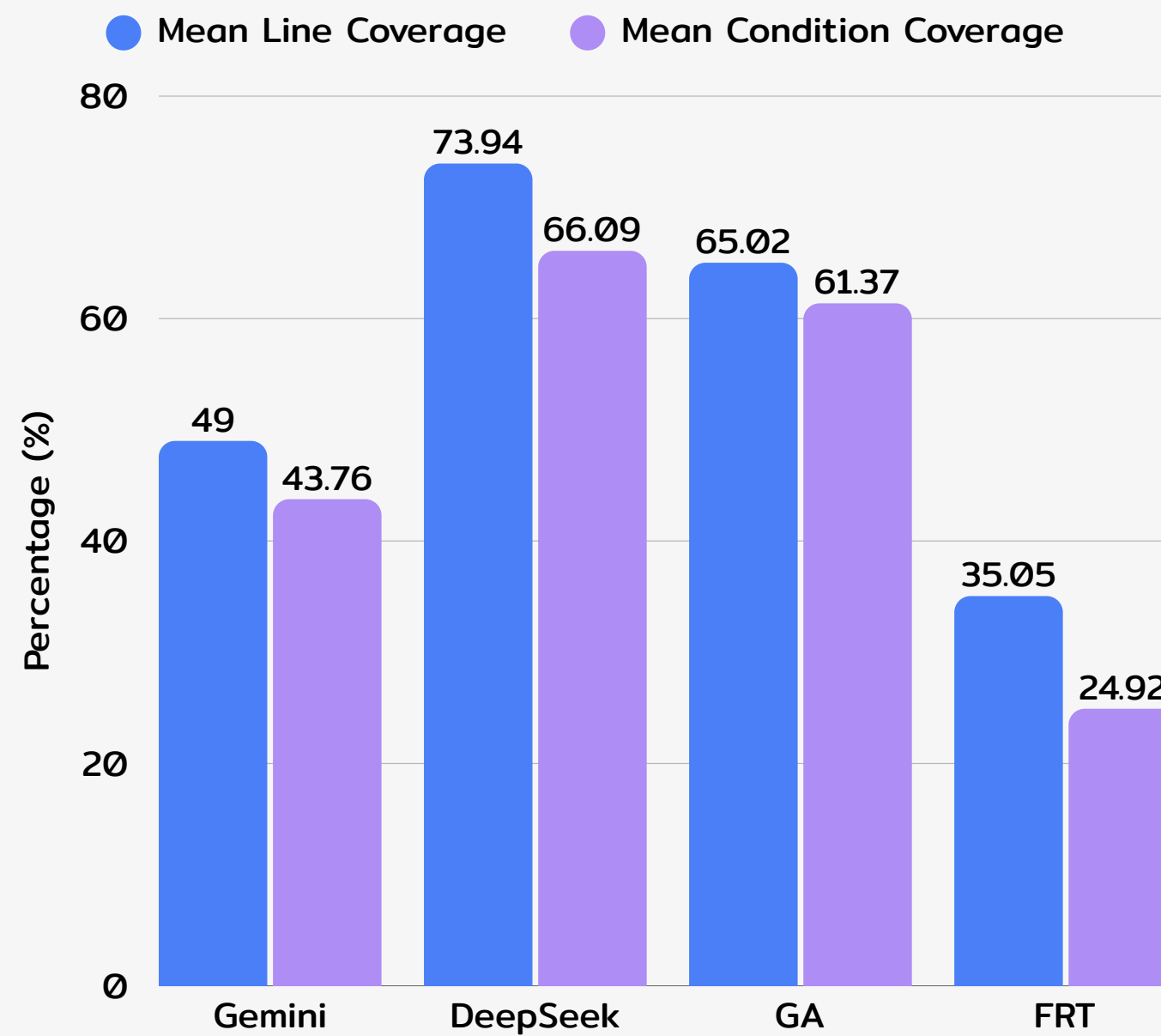
FAIL

ไม่สามารถสร้างชุดทดสอบของ
BUG ID นั้นๆได้เลย

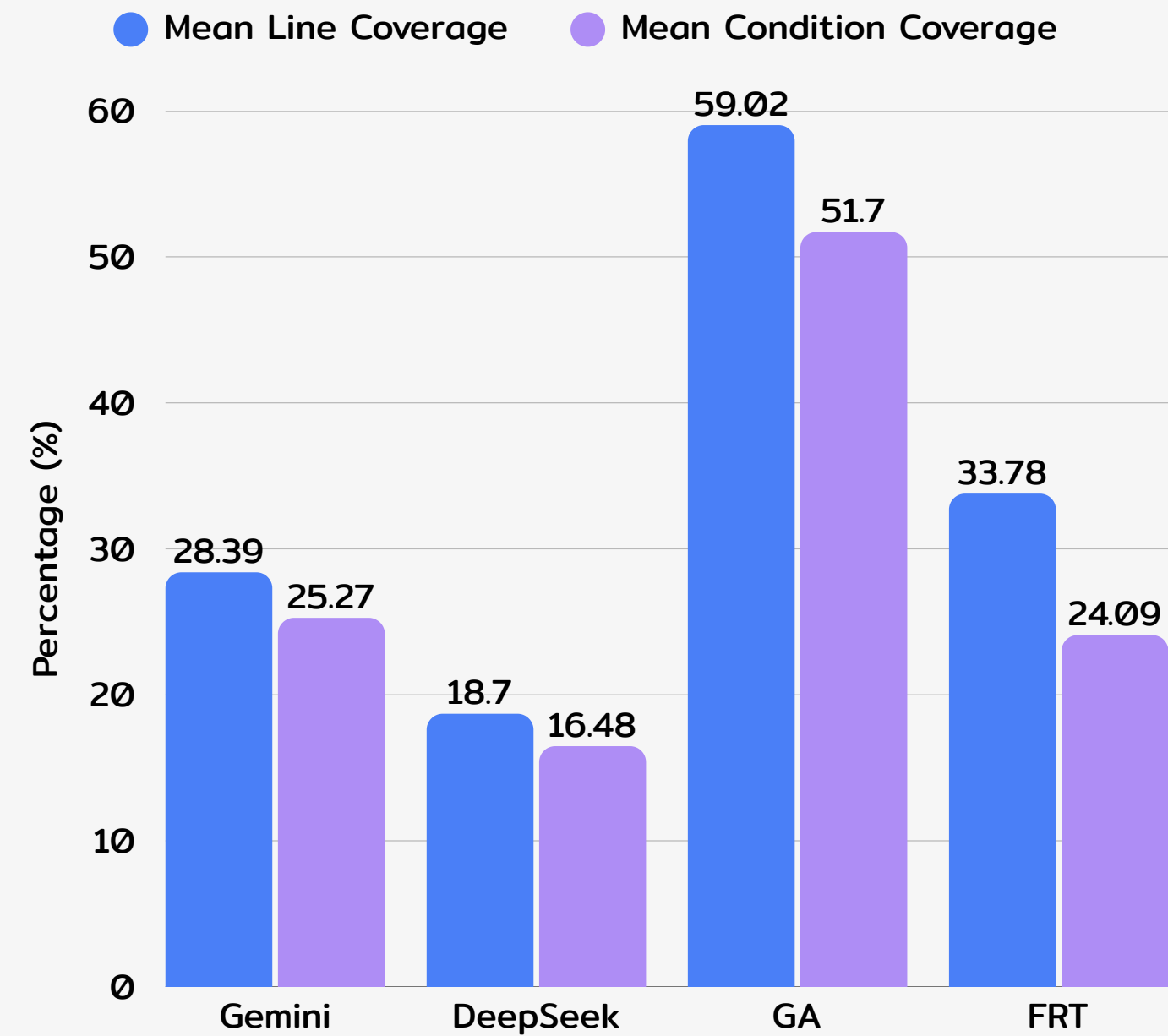
COVERAGE COMPARISON

Measured vs Overall Coverage

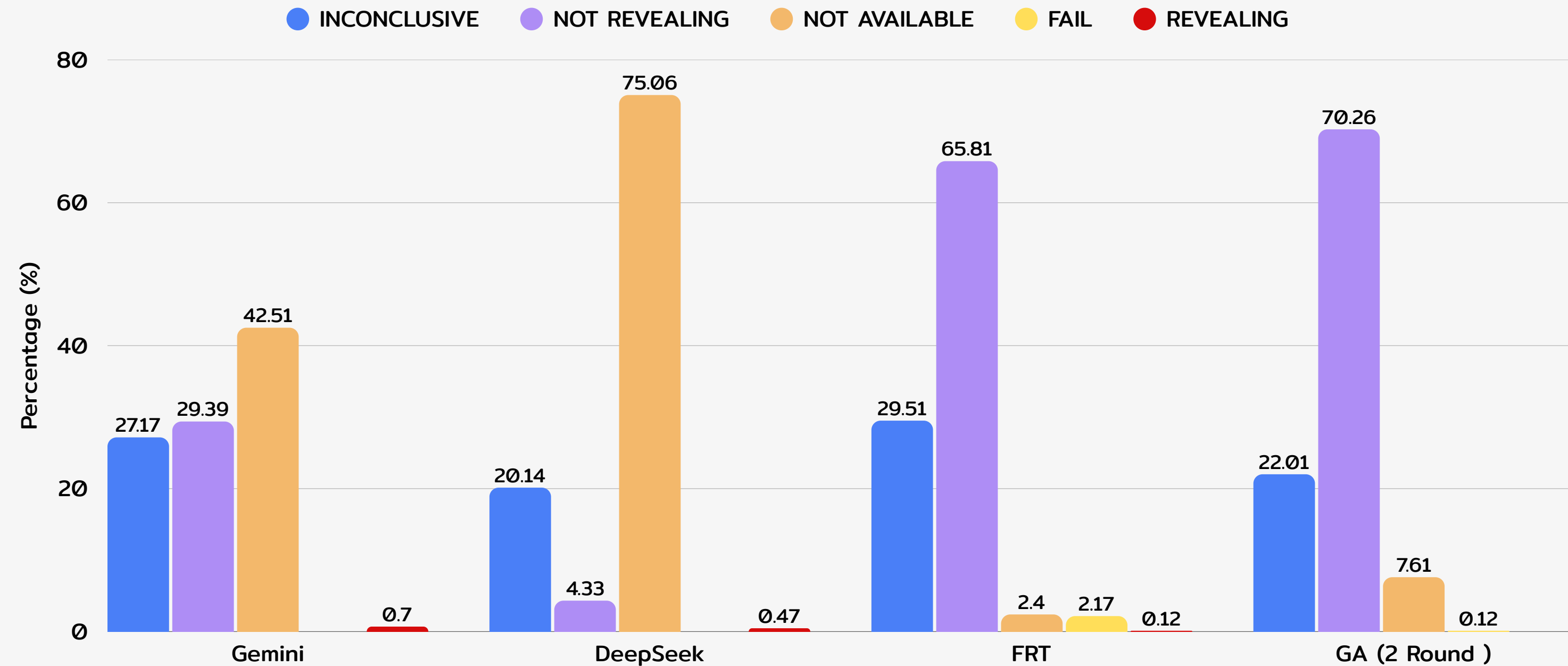
Measured



Overall



ผลการจำแนกสถานะทั้งหมด



ข้อดีและข้อจำกัดของแต่ละแนวทาง

GENETIC ALGORITHM (GA)

จุดเด่น: OVERALL COVERAGE สูงและเสถียร
ข้อจำกัด: ตรวจสอบบักไม่ได้เลย

1

Feedback-directed Random Testing (FRT)

จุดเด่น: รันได้เสถียรที่สุด
ข้อจำกัด: COVERAGE ต่ำที่สุดและตรวจสอบบักได้น้อย

2

DEEPSEEK-V4-FLASH

จุดเด่น: COVERAGE สูงที่สุด
ข้อจำกัด: NOT_AVAILABLE มากที่สุด

3

GEMINI 3.8 FLASH

จุดเด่น: ตรวจสอบบักได้ดีที่สุด
ข้อจำกัด: NOT_AVAILABLE สูง

4



ข้อจำกัดและอุปสรรคในการทดลอง

ประเด็นที่พบระหว่างการทดลอง

สภาพแวดล้อม

- ต้องเตรียม DEFECTS4J, JAVA และ LINUX/WSL
- ต้อง CHECKOUT + COMPILE ก่อนการทดลอง

การสร้าง TEST CASE

- AI ใช้เวลาและ TOKEN ต่างกันตามขนาด SOURCE CODE
- บางคลาสมีข้อมูลขนาดใหญ่ ทำให้ประมวลผลนาน

การรัน TEST SUITE

- TEST บางส่วน COMPILE หรือ RUN ไม่ผ่าน
- เกิดผล NOT_AVAILABLE / INCONCLUSIVE/FAIL

ข้อจำกัดของเครื่องมือ

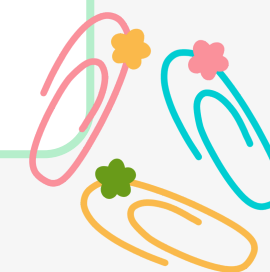
- AI พบปัญหา API, TOKEN และ GATEWAY TIMEOUT
- EVOSUITE/RANDOOOP ใช้ทรัพยากรเครื่องสูงเมื่อทดลองหลาย BUG ID



สรุปผลการทดลอง

จากการทดลองทั้ง 4 แนวทาง พบว่า ไม่มีเครื่องมือใดให้ผลดีที่สุดในทุกด้าน โดยแต่ละแนวทางมีจุดเด่นแตกต่างกัน

- ด้าน COVERAGE: DEEPSEEK มี MEASURED COVERAGE สูงที่สุด โดยมี LINE COVERAGE 73.94% แต่เมื่อพิจารณา OVERALL COVERAGE แล้ว GENETIC ALGORITHM (GA) ทำได้ดีที่สุด ที่ 59.02%
- ด้านการตรวจจับ BUG: GEMINI มีอัตรา REVEALING สูงที่สุดที่ 0.70% ตามด้วย DEEPSEEK 0.47%, FRT 0.12% และ GA 0.00%
- ด้านความเสถียร: GA และ FRT สามารถรับการทดลองได้สม่ำเสมอว่า AI โดย DEEPSEEK มี NOT_AVAILABLE สูงถึง 75.06%
- ดังนั้น COVERAGE สูงไม่ได้หมายความว่า จะตรวจจับ BUG ได้ดีที่สุด การเลือกเครื่องมือควรพิจารณาทั้ง COVERAGE, ความสามารถในการตรวจจับ BUG และความเสถียรในการรัน



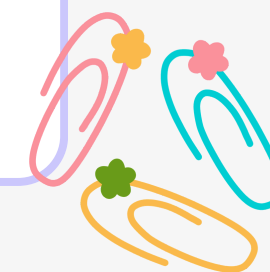
สิ่งที่ได้เรียนรู้และแนวทางการพัฒนาในอนาคต

สิ่งที่ได้เรียนรู้

- การประเมิน TEST SUITE ต้องดูทั้ง COVERAGE, การรันจริง และการตรวจจับ BUG
- COVERAGE สูงไม่ได้หมายความว่าตรวจจับ BUG ได้ดี
- คุณภาพของ ASSERTION มีผลต่อการตรวจพบข้อผิดพลาดโดยตรง
- แต่ละเครื่องมือมีจุดเด่นและข้อจำกัดแตกต่างกัน

แนวทางการพัฒนา

- ปรับปรุง PROMPT + CONTEXT สำหรับ AI เพื่อสร้าง TEST CASE ที่มีคุณภาพ
- วิเคราะห์สาเหตุของ NOT_AVAILABLE ให้ละเอียดขึ้น
- พัฒนาแนวทาง HYBRID TEST GENERATION ระหว่าง AUTOMATED TOOLS + AI
- เพิ่มจำนวนรอบการทดลองและใช้ การวิเคราะห์ทางสถิติ



CP353201 • SOFTWARE QUALITY ASSURANCE

THANK YOU

QUESTIONS & ANSWERS

